

Name: Mahitha Kalaga

UD ID: 101831115

Course Name: Artificial Intelligence

Project 3: Crawler Simulator by RL

1. Implementation Overview

In this project, I implemented the reinforcement learning logic for a crawling bot. The following key functions were integrated into the `RLearning.py` file:

chooseAction(self, r, c): Implemented the ϵ -greedy action selection algorithm. It correctly handles the exploration-exploitation tradeoff using the given 'epsilon' variable. It selects uniformly among random actions with a probability of 'epsilon', and greedily picks the action with the maximum Q-value otherwise, making sure to randomly break ties if multiple actions share the same maximum Q-value. Also properly bounds check the angles before making a final update.

onMonteCarlo(self): Implemented First-Visit Monte Carlo learning. For `self.steps` episodes, the agent generates a random-length trajectory, records the total return (horizontal distance traveled), and updates the Q-values based on the average returns iteratively by using an auxiliary visitation counts table self.N.

onTDLearning(self): Implemented TD SARSA learning. The agent updates its Q-values online at each step within an episode using the SARSA update rule ($Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A) - Q(S, A)]$).

onQLearning(self): Implemented Q-Learning. Similar to SARSA, but uses the max operator over next possible actions to update the Q-values offline at each step ($Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$).

2. Evaluation

Properly Implemented Components:

- The ϵ -greedy policy action choosing is done very efficiently since it performs robust boundary checking when crawling on states, so as to prevent movement beyond allowed angles.
- Monte Carlo learning uses the first visit strategy with running average calculations, all without the need for any parameter tuning.
- TD SARSA and Q-learning both successfully compute local rewards for each step in continuous motion.

Improvement Areas / Could Be Better:

- Due to the randomized lengths of trajectories, we do not have a predetermined terminal state, implying that our crawler continuously learns on an infinite horizon basis.
- In case we end up with extreme angles, where the crawler cannot move, it will spend some time doing nothing.

- Hyperparameters (alpha, gamma, and epsilon) can be optimized further through GUI sliders.

2. Screenshots





